

LEARNING MANAGEMENT SYSTEM

ABSTRACT

The AI-Powered Learning Management System (LMS) is a web-based application developed to enhance the teaching and learning experience by integrating artificial intelligence into traditional learning management functionalities. The system provides separate dashboards for teachers and students, ensuring role-based access to educational resources and management features.

Teachers can create classrooms, upload learning materials, and manage classroom content efficiently. Students can access the available classrooms, view uploaded documents, and utilize AI-powered learning tools generated from those documents. These tools include flashcard generation for active revision, quiz generation for self-assessment, an AI Assistant that answers questions based on the uploaded documents, and an AI Evaluator that analyzes handwritten notes and provides marks and feedback.

The application is developed using the MERN Stack, with Clerk handling secure authentication and role-based authorization. OpenAI is integrated to provide intelligent content generation and document-based question answering. By combining classroom management with AI-driven educational tools, the proposed system reduces teachers' workload while providing students with an interactive and personalized learning experience.

PO and PSO Addressed:

PO1,PO2,PO3,PO4,PO5,PO6,PO7,PO9,PO10,PO11,PSO1,PSO2 and PSO3.

INTRODUCTION

Learning Management Systems (LMS) have become an essential part of modern education by enabling institutions to distribute study materials, manage classrooms, and facilitate communication between teachers and students. Although existing LMS platforms provide basic classroom management and document sharing capabilities, they often lack intelligent features that can actively support the learning process.

Teachers usually spend significant time preparing study materials, creating quizzes, and evaluating students' work manually. Similarly, students often rely on repeated reading of documents and external resources to understand concepts and assess their knowledge. These limitations create the need for a smarter learning platform that can automate educational tasks and provide personalized learning support.

The proposed AI-Powered Learning Management System addresses these challenges by integrating artificial intelligence into a conventional LMS. Teachers can create classrooms and upload study materials, while students can use AI-generated flashcards, quizzes, document-based question answering, and handwritten note evaluation. These features improve learning efficiency, encourage self-assessment, and provide immediate feedback without requiring continuous teacher intervention.

The application is developed using the MERN Stack for building a scalable web application, Clerk for secure user authentication and role management, and OpenAI for generating intelligent educational content.

OBJECTIVES

The primary objective of the proposed AI-Powered Learning Management System is to develop an intelligent, secure, and user-friendly web application that enhances the teaching and learning experience through the integration of artificial intelligence. The system is designed to provide separate role-based dashboards for teachers and students, enabling teachers to create classrooms, upload study materials, and manage educational content efficiently. It also enables students to access classroom resources, view uploaded documents, and improve their learning through AI-generated flashcards, quizzes, and an AI Assistant capable of answering questions based on the uploaded documents. In addition, the system promotes self-learning by allowing students to upload images of their handwritten notes for AI-based evaluation, which provides marks and constructive feedback. By combining classroom management with AI-powered educational tools, the project aims to simplify academic activities, reduce teachers' workload, encourage interactive learning, and provide a personalized and effective digital learning environment.

MODULE DESCRIPTION

Module 1: User Authentication and Role-Based Dashboard

The system supports two categories of users: teachers and students. User authentication is implemented using Clerk, which provides secure login, registration, and session management.

After successful authentication, the system identifies the user's role. Teachers are redirected to the Teacher Dashboard, while students are redirected to the Student Dashboard. This role-based navigation ensures that users can only access features relevant to their responsibilities.

Module 2: Teacher Dashboard

The Teacher Dashboard acts as the primary workspace for teachers to manage classrooms and learning resources.

Teachers can create new classrooms, view previously created classrooms, and upload documents to individual classrooms. Each classroom serves as an independent learning space where study materials are organized systematically. Teachers can continuously update classroom content by adding new documents whenever required.

Module 3: Student Dashboard

The Student Dashboard provides students with access to all classrooms created by teachers.

Students can browse the available classrooms and select a classroom of interest. After entering a classroom, they can view all the study documents uploaded by the teacher. This organized structure enables students to easily access learning materials for different subjects or courses.

Module 4: Document Management

The document management module allows teachers to upload study materials to their classrooms. These documents form the primary source of information for all AI-powered learning features available in the system.

Students can access the uploaded documents through their respective classrooms. Selecting a document opens the content page, where various AI-powered learning tools become available. This module ensures that educational resources remain organized and accessible while serving as the foundation for AI-generated content.

Module 5: Flashcard Generation

The Flashcard module automatically generates interactive flashcards from the uploaded document using OpenAI.

After processing the document, important concepts are converted into question-and-answer flashcards. Students can review these flashcards to strengthen their understanding and improve memory retention through active recall. This eliminates the need for manually creating revision materials and provides an efficient way to revise key concepts.

Module 6: Quiz Generation

The Quiz module creates multiple-choice questions based on the uploaded study material.

Using OpenAI, the document is analyzed and transformed into objective questions with multiple answer options. Students can attempt the quiz to evaluate their understanding of the document. After completion, the system automatically calculates the score and displays the results, allowing students to identify areas requiring improvement.

Module 7: AI Assistant

The AI Assistant functions as a virtual tutor that answers questions related to the uploaded document.

Students and teachers can ask questions in natural language, and the assistant generates responses based solely on the document's content. This enables users to clarify doubts instantly without searching through lengthy study materials. By focusing only on the uploaded document, the assistant provides relevant and context-aware answers that support effective learning.

Module 8: AI Evaluator

The AI Evaluator enables students to evaluate their handwritten notes using artificial intelligence.

Students upload an image of their handwritten notes through the evaluator interface. The system analyzes the uploaded image and compares its content with the original study document. Based on the relevance, completeness, and accuracy of the notes, the AI generates marks and detailed feedback. This feature encourages self-learning and helps students identify missing concepts before examinations.

SOFTWARE REQUIREMENTS

React.js

React.js is used to develop the frontend of the application, providing a responsive and interactive user interface. Its component-based architecture enables efficient rendering and improves the overall user experience.

Node.js

Node.js serves as the runtime environment for executing JavaScript on the server side. It enables the application to handle multiple client requests efficiently and supports scalable backend development.

Express.js

Express.js is used to build the backend server and RESTful APIs. It simplifies routing, request handling, middleware integration, and communication between the frontend and the database.

MongoDB Atlas

MongoDB Atlas is the cloud-based NoSQL database used to store user information, classroom details, uploaded documents, flashcards, quizzes, and other application data securely and efficiently.

Clerk Authentication

Clerk provides secure user authentication and authorization services. It manages user registration, login, session management, and role-based access control for both teachers and students.

OpenAI API

The OpenAI API powers the intelligent features of the application by generating flashcards, quizzes, document-based question answering, and handwritten note evaluation. It enables the system to provide personalized and interactive learning experiences.

Visual Studio Code

Visual Studio Code is the primary development environment used to write, edit, and debug the application's source code. It provides various extensions and tools that improve development productivity.

CODE

server.js:

```
import express from "express"

import dotenv from "dotenv"

import cors from "cors"

import connectDb from "../config/db.js";

import { clerkMiddleware } from "@clerk/express";

import { userRouter } from "../router/userRouter.js";

import { classRouter } from "../router/classRouter.js";

import { contentRouter } from "../router/contentRouter.js";

const app = express()
```

```
dotenv.config()

const PORT = process.env.PORT;

//middleware

app.use(express.json());

app.use(cors());

app.use(clerkMiddleware());

app.get("/",async(req,res)=>{

    res.send("Server is Live")

})

app.use("/api",userRouter);

app.use("/api",classRouter);

app.use("/api",contentRouter);

connectDb();

app.listen(PORT,()=>{

    console.log(`Server running on PORT http://localhost:${PORT}`);

})
```

/model/classModel.js:

```
import mongoose from "mongoose";

const classSchema = new mongoose.Schema({

    userId : {type:String,required:true},

    name:{type:String,required:true},
```

```
    mentorName : {type:String,required:true},  
    courseName : {type:String,required:true},  
    courseDuration : {type:String,required:true},  
    documents : {type:[Object],default:[],required:true}  
  })  
  
const classModel = mongoose.model("class",classSchema) ||  
mongoose.models.class;  
  
export {classModel}
```

/model/contentModel.js:

```
import mongoose from "mongoose";  
  
const contentSchema = new mongoose.Schema({  
  classRoomId : {type:String,required:true},  
  documentId:{type:String,required:true},  
  documentText:{type:String,required:true},  
  content : {type:Object,required:true}  
})  
  
const contentModel = mongoose.model("content",contentSchema) ||  
mongoose.models.content;  
  
export {contentModel}
```

/model/userModel.js:

```
import mongoose from "mongoose";  
  
const userSchema = new mongoose.Schema({
```

```
    userId : {type:String,required:true},
    name:{type:String,required:true},
    email:{type:String,required:true},
    role : {type:String,required:true}
  })

const userModel = mongoose.model("user",userSchema) || mongoose.models.user;
export{userModel}
```

/controller/classController.js:

```
import { classModel } from "../model/classModel.js";
import fs from "fs";
import pdf from "pdf-parse/lib/pdf-parse.js";
import OpenAI from "openai";
import { v4 as uuidv4 } from "uuid";
import { contentModel } from "../model/contentModel.js";
import dotenv from "dotenv";
dotenv.config();

const openai = new OpenAI({
  apiKey: process.env.GEMINI_API_KEY,
  baseURL: "https://generativelanguage.googleapis.com/v1beta/openai/"
});

const createClassroom = async(req,res) => {
```

```

try{
    const {name,mentorName,courseName,courseDuration} = req.body;
    const userId = req.auth.userId
    const classRoom = new classModel({
        userId:userId,
        name:name,
        mentorName:mentorName,
        courseName:courseName,
        courseDuration:courseDuration,
    })
    await classRoom.save();
    res.json({success:true,message:"Classroom created"})
}
catch(error){
    console.log(error);
    res.json({success:false,message:"Error"}})}

const fetchClassroom = async(req,res) => {
    try{
        const userId = req.auth.userId;
        const classroom = await classModel.find({userId});
        res.json({success:true,classroom});
    }
    catch(error){
        console.log(error);
        res.json({success:false,message:"Error"}})}
}

```

```

    }
    catch(error){
        console.log(error);
        res.json({success:false,message:"Error"})
    }
}

const addDocument = async(req,res) => {
    try{
        const {id,description,name} = req.body ;
        const document = req.file;
        if(document.size > 5*1024*1024){
            return res.json({success:false,message:"File size exceeds allowed size
(5MB)."})
        }
        const databuffer = fs.readFileSync(document.path);
        const pdfData = await pdf(databuffer);
        const unique = uuidv4();

        await classModel.findByIdAndUpdate(id,{
            $push : {documents : {
                documentId: unique,
                name:name,
                description: description,

```

```
documentText: pdfData.text
```

```
  }  });
```

```
const prompt = `You are an AI that generates study materials.
```

```
From the given document text, generate:
```

1. Flashcards
2. Quiz (MCQs with 4 options and correct answer index)
3. Important Questions

```
STRICT RULES:
```

- Output ONLY valid JSON
- Do NOT include explanations or extra text
- Each quiz must have exactly 4 options
- correctAnswer MUST be the index (0,1,2,3)
- Ensure the index matches the correct option

```
JSON FORMAT:
```

```
{  
  "flashcards": [{ "question": "", "answer": "" }],  
  "quiz": [  
    {  
      "question": "",  
      "options": [ "", "", "", "" ],  
      "correctAnswer": 0
```

```
    }  
  ],  
  "importantQuestions": [{"question": ""}]  
}
```

DOCUMENT:

"""

\${pdfData.text}

"""

```
    const response = await openai.chat.completions.create({  
      model: "gemini-3-flash-preview",  
      messages: [  
        {  
          role: "user",  
          content: prompt,  
        },  
      ],  
      temperature: 0.7,  
    });  
  
    const content = response.choices[0].message.content;  
    const cleaned = content.replace(/```json|```/g, "").trim();  
    const json = JSON.parse(cleaned);
```

```
const documentContent = new contentModel({
  classRoomId: id,
  documentId:unique,
  documentText: pdfData.text,
  content:json
})

await documentContent.save();

res.json({success:true,message:"Documents Added and the content is
generated"});
}

catch(error){
  console.log(error);
  res.json({success:false,message:"Error"});
}
}

const getClassroom = async(req,res) => {
  try{
    const classDocument = await classModel.find({});
    res.json({success:true,data:classDocument});
  }
  catch(error){
    console.log(error);
```

```

        res.json({success:false,message:"Error"});}}

const deleteClassroom = async(req,res) => {

  try{

    const {id} = req.body;

    await classModel.findByIdAndDelete(id);

    await contentModel.deleteMany({classRoomId:id});

    res.json({success:true,message:"Classroom deleted"});

  }

  catch(error){

    console.log(error);

    res.json({success:false,message:"Error"});

  }}

const deleteDocument = async(req,res) => {

  try{

    const {id,doc} = req.body;

    await classModel.findByIdAndUpdate(id,{ $pull:{ documents:doc } });

    await contentModel.deleteMany({documentId:doc.documentId});

    res.json({success:true,message:"Document Deleted"});

  }

  catch(error){

    console.log(error);

```

```
res.json({success:false,message:"Error"});}}
```

```
export  
{createClassroom,fetchClassroom,getClassroom,addDocument,deleteClassroom,de  
leteDocument}
```

/router/classRouter.js:

```
import express from "express";
```

```
import { requireAuth } from "@clerk/express";
```

```
import { addDocument, createClassroom, deleteClassroom, deleteDocument,  
fetchClassroom, getClassroom } from "../controller/classController.js";
```

```
import multer from "multer"
```

```
const classRouter = express.Router();
```

```
const storage = multer.diskStorage({
```

```
  destination: 'uploads/',
```

```
  filename : (req,file,cb) => {
```

```
    return cb(null, `${Date.now()+file.originalname}`);
```

```
  }
```

```
});
```

```
const uploads = multer({storage});
```

```
classRouter.post("/create-classroom",requireAuth(),createClassroom);
```

```
classRouter.get("/fetch-classroom",requireAuth(),fetchClassroom);
```

```
classRouter.post("/add-document",uploads.single("document"),addDocument);
```

```
classRouter.post("/delete-class",deleteClassroom);
```

```
classRouter.post("/delete-document",deleteDocument);
```

```
classRouter.get("/all",getClassroom)
```

```
export{classRouter}
```

/router/contentRouter.js:

```
import { assistantEvaluator, assistantResponse, fetchAssignment, fetchFlashcard, fetchImportant } from "../controller/contentController.js";
```

```
import express from "express";
```

```
const contentRouter = express.Router();
```

```
contentRouter.post("/flashcards",fetchFlashcard);
```

```
contentRouter.post("/imp",fetchImportant);
```

```
contentRouter.post("/assignment",fetchAssignment);
```

```
contentRouter.post("/assistant",assistantResponse);
```

```
contentRouter.post("/evaluator",assistantEvaluator);
```

```
export {contentRouter};
```

/router/userRouter.js:

```
import express from "express";
```

```
import { requireAuth } from "@clerk/express";
```

```
import { setRole } from "../controller/userController.js";
```

```
const userRouter = express.Router();
```

```
userRouter.post("/set-role",requireAuth(),setRole);
```

```
export{userRouter}
```

Navbar.jsx:

```
import React from 'react'

import logo from "/src/assets/learnlink.png"

import "./Navbar.css"

import { useLocation, useNavigate } from 'react-router-dom'

import { UserButton, useUser } from '@clerk/clerk-react'

import { useEffect } from 'react'

const Navbar = () => {

  const navigate = useNavigate();

  const location = useLocation();

  const {isSignedIn,isLoaded,user} = useUser();

  useEffect(() => {

    if (!isLoaded || !isSignedIn) return;

    if(location.pathname !=="/") return;

    const role = user?.publicMetadata?.role;

    if (role === "teacher") {

      navigate("/teacher-dashboard");}

    if (role === "student") {

      navigate("/student-dashboard");

    }

  }, [isLoaded, isSignedIn, user,navigate]);

  return (
```

```

<div className='nav-bar'>

  <img src={logo} className='logo' />

  { !isSignedIn ? <div className='sign-in-up-button'>

    <button className='sign-up-button' onClick={()=>{navigate("/signup")}}>
    >Get Started</button>

    <button className='sign-in-button'
    onClick={()=>{navigate("/signin")}}>Login</button>

    </div> : <UserButton /> }

  </div>))}

```

export default Navbar

TeacherDashboard.jsx:

```

import React from 'react'

import './TeacherDashboard.css'

import Navbar from '../components/Navbar/Navbar.jsx'

import { useUser } from '@clerk/clerk-react'

import { useNavigate } from 'react-router-dom'

const TeacherDashboard = () => {

  const navigate = useNavigate();

  const {user} = useUser();

  const username = user?.firstName;

  return (

    <div className='teacher-dashboard'>

```

```

<Navbar />

<h1 className='teacher-dashboard-heading'>Welcome {username},</h1>

<div className='teacher-features'>

  <div className='teacher-feature' onClick={()=>{
    navigate("/create-classroom")
  }} style={{backgroundColor:"orange"}} >

    <h2>Create Classroom</h2>

    <p>Teacher can create their own classrooms and assign courses here.</p>
  </div>

  <div className='teacher-feature' onClick={()=>{
    navigate("/classroom")}} style={{backgroundColor:"violet"}} >

    <h2>My Classroom</h2>

    <p>Teacher can view how many classes they are maintaining.</p>
  </div>

</div>

)
}

export default TeacherDashboard

```

StudentDashboard.jsx:

```
import React, { useContext, useState } from 'react'
```

```
import { DataContext } from '../dataContext.jsx'

import loadersvg from "/src/assets/loader.svg"

import { useEffect } from 'react';

import Navbar from '../components/Navbar/Navbar.jsx';

import "./StudentDashboard.css"

import { useNavigate, useSearchParams } from 'react-router-dom';

import { ToastContainer, toast } from "react-toastify";

import axios from 'axios';

const StudentDashboard = () => {

  const {loader,Classroom,sselectedclass,allClassroom,classList} =
  useContext(DataContext);

  const navigate = useNavigate();

  const [search,setsearch] = useState("");

  const [filteredClass,setfilteredClass] = useState(classList);

  useEffect(()=>{

    allClassroom();

    if(search){

      setfilteredClass(classList.filter((item)=>(item._id.includes(search))));

    }

    else{

      setfilteredClass(classList);

    }

  })
```

```

},[search,classList]);

return (

<div>

  <Navbar/>

  {!!loader ? <div>

    <h1 className='classroom-heading'>Classrooms</h1>

    <div className='input-tag'>

      <label>Enter Classroom Id:</label>

      <input type='text' onChange={(e)=>{setsearch(e.target.value);}} />

    </div>

    <div className='classroom-list' >

      {filteredClass?.map((item,index)=>(

        <div className='classroom-card' key={index} >

          <h1>{item.name}</h1>

          <p>id:{item._id}</p>

          <p><b>Mentor name:</b> {item.mentorName}</p>

          <p><b>Course name:</b> {item.courseName}</p>

          <p><b>Duration:</b> {item.courseDuration}</p>

          <button onClick={()=>{navigate(`/stud-documents/${item._id}`)}}
style={{marginLeft:"10px"}} >View Documents</button>

        </div>

      )

    )

  )
}

```

```

        </div>

        </div> :

        <img src={loadersvg} className='loader' />

    }

    <ToastContainer/>

</div>

)}

export default StudentDashboard

```

ContentDashboard.jsx:

```

import React from 'react'

import { useNavigate, useParams } from 'react-router-dom'

import Navbar from '../components/Navbar/Navbar.jsx'

import { useUser } from '@clerk/clerk-react'

import './ContentDashboard.css'

const ContentDashboard = () => {

    const navigate = useNavigate();

    const {user} = useUser();

    const username = user?.firstName;

    const {documentId} = useParams();

    return (

```

```

<div className='content-dashboard'>

  <Navbar />

  <h1 className='content-dashboard-heading'>Welcome {username},</h1>

  <div className='content-features'>

    <div className='content-feature' onClick={()=>{

      navigate(`/content/flashcard/${documentId}`)

    }} style={{backgroundColor:"orange"}} >

      <h2>Flashcards</h2>

      <p>A set of flashcards which are related to the document is available
here.</p>

    </div>

    <div className='content-feature' onClick={()=>{

      navigate(`/content/assignment/${documentId}`)}}
style={{backgroundColor:"violet"}} >

      <h2>Quiz</h2>

      <p>Students can solve the AI generated quiz and practice themselves.</p>

    </div>

    <div className='content-feature' style={{backgroundColor:"green"}}
onClick={()=>{navigate(`/content/important/${documentId}`)}} >

      <h2>Important Questions</h2>

      <p>Document is completely analyzed and the important questions are
here.</p>

    </div>

```

```
<div className='content-feature' style={{backgroundColor:"red"}}  
onClick={()=>{navigate(`/content/assistant/${documentId}`)}} >
```

```
<h2>AI Assistant</h2>
```

```
<p>Students can ask any doubt about the documents here.</p>
```

```
</div>
```

```
<div className='content-feature' style={{backgroundColor:"purple"}}  
onClick={()=>{navigate(`/content/evaluator/${documentId}`)}} >
```

```
<h2>AI Evaluator</h2>
```

```
<p>Teachers can evaluate the answers they have.</p>
```

```
</div>
```

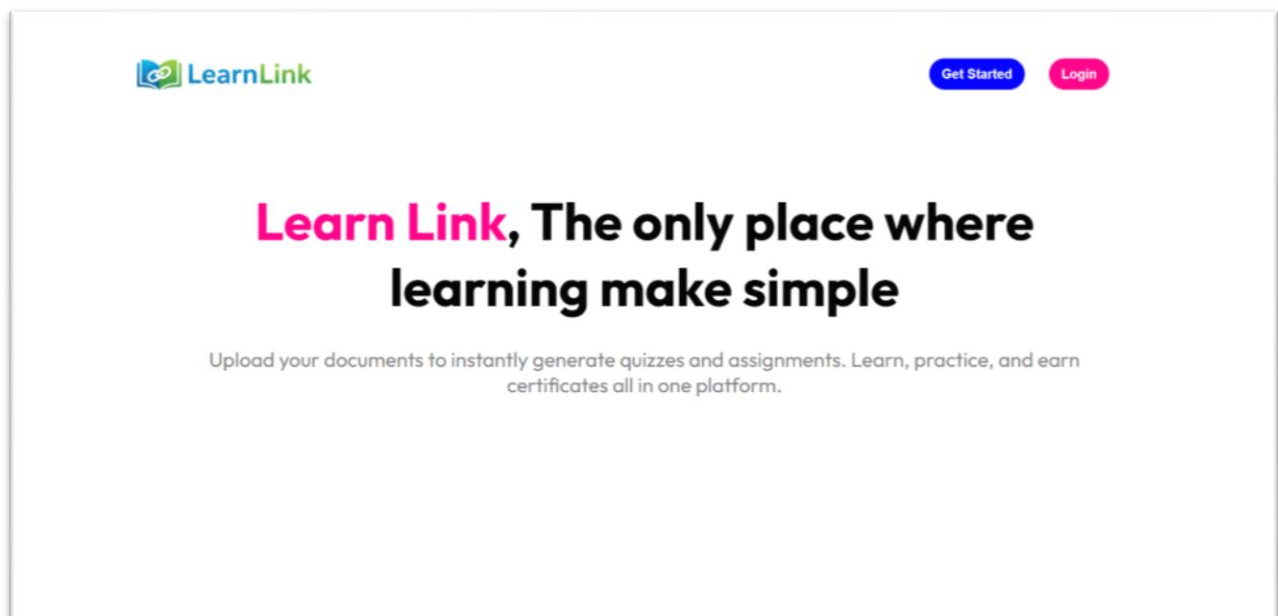
```
</div>
```

```
</div>))
```

```
export default ContentDashboard
```

APPLICATION SCREENSHOTS

Home Page:



Select Role Page:

Select your role?

I am a Teacher

I am a Student

Submit

Teacher Dashboard Page:

 LearnLink

M

Welcome Muthu Maheswaran M,

Create Classroom

Teacher can create their own classrooms and assign courses here.

My Classroom

Teacher can view how many classes they are maintaining.

Create Classroom Page:



Classroom creation

Enter Classroom Name
Web Development

Enter Mentor Name
ABC

Enter Course Name
Web Development Essentials

Enter Course Duration
20 days

Submit

Student Dashboard Page:



Classrooms

Enter Classroom Id:

Web Development

id:6a57bc02fc5f2664b94032e0

Mentor name: ABC

Course name: Web Development Essentials

Duration: 20days

View Documents

Content Page:

Welcome Muthu Maheswaran M,

Flashcards

A set of flashcards which are related to the document is available here.

Quiz

Students can solve the AI generated quiz and practice themselves.

Important Questions

Document is completely analyzed and the important questions are here.

AI Assistant

Students can ask any doubt about the documents here.

AI Evaluator

Teachers can evaluate the answers they have.

Flashcard Page:

Flashcards

←

What is source code?

→

Quiz Page:

Quiz

Which programming paradigm treats every operation as self-sufficient and is strongly influenced by mathematical formalism?

Procedural programming

Object-oriented programming

Functional programming

Declarative programming

Which HTML element is used to specify the character encoding for the document?

<charset>

<meta charset='utf-8'>

<link encoding='utf-8'>

<script type='encoding'>

In CSS, which selector matches only the 'li' elements that are children of an element with the ID 'content'?

.content li

li#content

#content li

li > .content

Which JavaScript operator checks for both equal value and equal type?

==

=

!=

===

What is the purpose of 'npm init' in a Node.js project?

To install all dependencies listed in node_modules

To generate a package.json file describing the project

To start the HTTP server

To compile JavaScript into machine code

Important Question Page:

Important Questions

- What is the difference between a compiled language and an interpreted language?
- Explain the concept of 'Responsive Web Design' and how media queries are utilized.
- How does the 'strict comparison' (===) differ from 'loose comparison' (==) in JavaScript?
- What is SQL injection, and how can using placeholders in SQLite prevent it?
- Describe the purpose and common usage of the 'target' attribute in HTML anchors.

AI Assistant Page:

AI Assistant

You can ask question about the document and the AI assistant will reply.Note:Copy the results as soon as possible,they are not saved to see later.

Which HTML element is used to specify the character en

→

The `<meta>` element is used to specify the character encoding for an HTML document, typically using the `charset` attribute (for example: `<meta charset="utf-8">`).

Copy

AI Evaluator Page:

AI Evaluator

Upload your images below and evaluate your answers

Choose image

Easy

Medium

Hard

Get marks

correctedText: 1. What is the difference between source code and machine code? Answer: Source code is the human-readable program written by developers using programming languages such as C, JavaScript, or Python. It is easy to understand and modify. Machine code, on the other hand, is a binary language (0s and 1s) that the computer's processor executes directly. A compiler or interpreter converts source code into machine code before execution. 2. What is the client-server architecture in web development? Answer: Client-server architecture is a model where the client (such as a web browser) sends requests to a server over a network. The server processes the request, retrieves any required data, and sends a response back to the client. The client is responsible for displaying the information to the user, while the server handles business logic, database operations, and content generation. score: 9/10 evaluation: The student provides highly accurate and well-structured answers that align closely with the provided reference document. For the first question, the student correctly identifies the nature of source code (human-readable, high-level) versus machine code (binary, processor-executable) and correctly identifies the role of compilers and interpreters as the conversion mechanism. For the second question, the student accurately describes the request-response cycle and the distinct responsibilities of the client (UI/display) and the server (logic, database, generation). Under "HARD" difficulty evaluation, a point is deducted because the student omitted the specific communication protocol (HTTP) which the reference document emphasizes as the standard for these web-based interactions. missingPoints: * The student failed to mention the specific protocol used for web client-server communication (HTTP/HTTPS), which is a key knowledge area in the reference (Objective 031.3). * In the distinction between codes, there was no mention of "bytecode" or "runtimes" (intermediate formats like Java uses), which the reference highlights as a specific middle-ground between source and machine code. * For the client-server model, the student did not mention the "Rendering

FUTURE ENHANCEMENT

The proposed system can be extended with several advanced features to further improve the learning experience. A mobile application can be developed using React Native to provide easy access from smartphones and tablets. Real-time classroom communication through chat and video conferencing can be integrated to enhance interaction between teachers and students. AI-based progress analytics can be introduced to monitor student performance and recommend personalized learning paths. Voice-based question answering and multilingual support can make the system more accessible to a wider audience. Gamification features such as badges, leaderboards, and achievement tracking can increase student engagement and motivation. Additional capabilities like assignment submission, plagiarism detection, attendance management, and integration with cloud storage platforms can further enhance the functionality of the learning management system.

CONCLUSION

The AI-Powered Learning Management System successfully combines classroom management with artificial intelligence to create a modern and interactive educational platform. By providing separate dashboards for teachers and students, the system ensures secure role-based access while maintaining an organized learning environment.

Teachers can efficiently manage classrooms and upload study materials, while students benefit from AI-generated flashcards, quizzes, document-based question answering, and handwritten note evaluation. These intelligent features reduce the manual effort involved in creating learning resources and provide immediate feedback that supports effective self-learning.

Developed using the MERN Stack, Clerk Authentication, and OpenAI, the application demonstrates how artificial intelligence can significantly enhance digital education. The system improves teaching efficiency, encourages active student participation, and offers a scalable foundation for future AI-driven educational innovations.